

CURVE METRICS ULTIMATE INTERVIEW GUIDE

HOW TO USE THIS GUIDE

This guide is designed to help you explain the project at multiple levels:

- ? short answers for fast interview responses
- ? medium-depth answers for follow-up rounds
- ? deep architectural answers for senior and system design interviews
- ? project-specific questions tied to what is actually in the codebase
- ? interviewer-style questions based only on what you say out loud

Best practice in interviews:

1. Start high-level.
2. Explain responsibilities before implementation details.
3. Say why a design exists, not just what it does.
4. Mention one tradeoff and one future improvement.
5. Be honest about weaknesses.

30-SECOND PROJECT SUMMARY

This project is a CMS-driven full stack platform with three main parts: an admin app for content authoring and publishing, a shared contract layer for layout/block normalization and validation, and a public Next.js website that server-renders published content. The most important architectural decision is a revision-based page workflow, so drafts, preview, restore, and publishing are all first-class instead of directly editing live content.

1-MINUTE PROJECT SUMMARY

The platform is split into 'admin', 'shared', and 'curvemetricswebsite'. The admin app is where internal users manage pages, media, templates, and publishing workflows. The shared layer defines canonical schemas, block contracts, normalization logic, and runtime validation so preview and production interpret content the same way. The public website is intentionally thin: it fetches a validated page bundle from the admin content API, derives a render view, and server-renders the final page. The strongest part of the design is the revision-first page lifecycle. The biggest weakness is that public delivery still depends on the admin/content runtime at request time, which creates scalability and availability concerns.

2-MINUTE ARCHITECTURE ANSWER

I would explain the system as a CMS-style platform built around content lifecycle safety. The admin app owns writes and workflows like edit, autosave, preview, restore, and publish.

Instead of storing only the current page state, the system stores revisions, so draft content and published content are separate. The shared layer owns the canonical meaning of blocks and layouts. It handles normalization, validation, and bundle assembly. The public site does not interpret raw CMS data directly. It fetches a validated page render bundle and renders it through SSR. That separation is intentional because it reduces preview-versus-production drift. The main tradeoff is increased application-level complexity, but the benefit is much stronger control over content correctness and publishing safety.

PROJECT SCOPE AND BUSINESS PURPOSE

WHAT THE SYSTEM SOLVES

- ? lets internal users build and manage a website without touching raw code
- ? supports draft editing without affecting the live site
- ? allows preview before publish
- ? preserves content history through revisions
- ? keeps public rendering consistent through a canonical content contract

MAIN USER TYPES

- ? admin/editor users
- ? public visitors

MAIN USER JOURNEYS

ADMIN JOURNEY

1. log in
2. choose a page
3. edit layout/content
4. autosave or manually save draft
5. preview draft
6. publish approved revision

PUBLIC JOURNEY

1. request route on website
2. server fetches published content
3. page is rendered server-side
4. final HTML is sent to browser

FULL SYSTEM ARCHITECTURE

MAIN APPLICATIONS

'ADMIN'

Owns:

- ? page authoring
- ? autosave
- ? version history
- ? publish workflow
- ? templates
- ? media upload
- ? auth-protected management APIs

'SHARED'

Owns:

- ? block registry
- ? layout normalization
- ? canonical render contracts
- ? SEO normalization
- ? runtime validation
- ? page view shaping

'CURVEMETRICSWEBSITE'

Owns:

- ? public route handling
- ? SSR page rendering
- ? thin content fetch boundary
- ? public render shell

CORE DESIGN RULE

The system tries to enforce:

'admin writes -> shared interprets -> website renders'

That is the core architectural story of the whole project.

WHY THIS ARCHITECTURE EXISTS

Without a shared interpretation layer:

- ? preview and public render would drift
- ? blocks would accumulate duplicate behavior
- ? frontend components would start repairing CMS data
- ? debugging would become much harder

MAIN ARCHITECTURAL TRADEOFFS

Benefits:

- ? strong content consistency
- ? safer publishing workflow
- ? reusable content contract
- ? easier reasoning about draft vs live state

Costs:

- ? more backend and shared-layer complexity
- ? tighter contract coordination across apps
- ? higher implementation effort than simple CRUD CMS

FRONTEND ARCHITECTURE

FRONTEND SPLIT

There are really two frontend systems:

- ? admin editor frontend
- ? public website frontend

ADMIN FRONTEND GOALS

- ? manage complex local editing state
- ? support drag and drop
- ? support undo/redo
- ? support autosave
- ? support preview/publish flows

PUBLIC FRONTEND GOALS

- ? stay thin
- ? render validated content
- ? preserve SEO via SSR
- ? avoid becoming a second content interpretation layer

ADMIN COMPONENT HIERARCHY

High level:

- ? route
- ? error boundary
- ? page editor shell
- ? toolbar
- ? left sidebar
- ? canvas
- ? property panel
- ? version/template/json modals

PUBLIC COMPONENT HIERARCHY

High level:

- ? route
- ? SSR fetch layer
- ? public page shell
- ? header
- ? banner
- ? page sections
- ? footer

ADMIN RENDERING FLOW

1. load selected page and revision-backed layout
2. normalize layout into editor-compatible shape
3. store it in local state
4. render sections/rows/columns/components
5. resolve block renderer through registry
6. update local state on edits
7. persist through autosave/manual save

PUBLIC RENDERING FLOW

1. fetch page bundle from admin content API
2. validate bundle
3. derive public page view
4. render shell and block tree
5. send SSR HTML response

STATE MANAGEMENT STRATEGY

The real editor state strategy is:

- ? local 'useState'
- ? workflow-oriented custom hooks
- ? refs for async coordination
- ? Context for app-wide concerns, not core layout state

Main custom hooks:

- ? page data loading and persistence
- ? layout mutation logic
- ? undo/redo
- ? autosave
- ? UI shell state

WHY THIS STATE STRATEGY WAS CHOSEN

- ? fast to iterate
- ? editor is a self-contained domain
- ? reduced global store overhead
- ? behavior grouped by workflow

WEAKNESSES OF CURRENT FRONTEND STATE MODEL

- ? large top-level editor component
- ? heavy prop orchestration
- ? broad re-render potential
- ? difficult long-term scalability if feature count grows

DRAG-AND-DROP HANDLING

- ? based on '@dnd-kit'
- ? drag metadata encodes layout location
- ? mutations happen in layout action hooks
- ? nested structures make the logic significantly harder than flat sorting

FRONTEND PERFORMANCE BOTTLENECKS

- ? full-layout object updates
- ? snapshot-based undo/redo
- ? JSON serialization for autosave/history comparisons
- ? large editor shell owning too much state
- ? nested block rendering complexity

FRONTEND SCALABILITY CONCERNS

- ? harder onboarding for new engineers
- ? more block types increase mutation complexity
- ? limited lazy loading in heavy editor paths
- ? preview/public shell duplication risk

FRONTEND IMPROVEMENT IDEAS

- ? split editor into smaller state-owning containers
- ? lazy-load history/templates/media panels
- ? isolate section-level rendering
- ? reduce whole-layout invalidation
- ? converge preview and public shells further

BACKEND ARCHITECTURE

WHAT THE BACKEND ACTUALLY IS

This is not a classic standalone Express server. It is a Node.js backend built mostly using:

- ? Next.js route handlers / API routes
- ? service layer
- ? repository layer
- ? MySQL
- ? NextAuth
- ? shared validation contracts

BACKEND LAYERING

ROUTE HANDLERS

Own:

- ? request parsing
- ? auth boundary entry
- ? status codes
- ? JSON responses

MIDDLEWARE

Owns:

- ? route protection for admin paths

SERVICES

Own:

- ? save workflow
- ? publish workflow
- ? render-source assembly
- ? template application

REPOSITORIES

Own:

- ? page lookup
- ? revision lookup
- ? direct DB operations

SHARED CONTRACTS

Own:

- ? canonical layout meaning
- ? bundle validation
- ? block schemas/defaults

WHY SERVICE/REPOSITORY SEPARATION EXISTS

Because save, publish, preview, and public render are workflows, not plain CRUD operations.

MAIN BACKEND STRENGTH

The page pipeline is workflow-aware and contract-aware rather than just exposing database rows.

MAIN BACKEND WEAKNESS

Architecture is strongest for pages, but some other domains still rely on simpler generic CRUD patterns.

AUTHENTICATION AND AUTHORIZATION

CURRENT AUTHENTICATION MODEL

- ? NextAuth credentials provider
- ? session-based auth

- ? middleware for route groups
- ? server-side 'requireAdmin()' checks on sensitive APIs

WHY IT WAS LIKELY CHOSEN

- ? simple internal admin tool
- ? low setup cost
- ? easy integration with Next.js

STRENGTHS

- ? simple mental model
- ? server-side session checks
- ? protected authoring APIs

WEAKNESSES

- ? effectively admin-only model
- ? no mature user management
- ? no strong RBAC
- ? no robust audit trail

CURRENT AUTHORIZATION REALITY

This is closer to:

- ? authenticated admin required

than to:

- ? full multi-role RBAC

BETTER FUTURE RBAC MODEL

Suggested roles:

- ? editor
- ? publisher
- ? admin
- ? media manager

Suggested permissions:

- ? edit drafts
- ? publish pages
- ? restore revisions

- ? manage globals
- ? manage media

API DESIGN

API STYLE

The backend uses resource routes plus workflow-specific action routes.

Examples:

- ? '/api/pages'
- ? '/api/pages/[page_id]'
- ? '/api/pages/[page_id]/autosave'
- ? '/api/pages/[page_id]/publish'
- ? '/api/page/[slug]'
- ? '/api/versions/list'
- ? '/api/versions/restore'

WHY THIS API STYLE MAKES SENSE

Publishing, autosave, and restore are not simple CRUD updates. They are domain commands.

TRADEOFFS

Pros:

- ? clearer workflow semantics
- ? easier to express domain actions

Cons:

- ? less ?pure REST?
- ? more endpoint variety

API VERSIONING REALITY

There is not strong explicit external versioning like '/v1'.

Versioning is more implicit through:

- ? shared bundle schema versioning
- ? coordinated app evolution

FUTURE IMPROVEMENT

If external consumers or more independent clients are added:

- ? add API versioning
- ? formalize public contract guarantees

VALIDATION AND ERROR HANDLING

VALIDATION PHILOSOPHY

CMS content should be treated as untrusted input even if it comes from internal tools.

MAIN VALIDATION POINTS

- ? request body parsing
- ? save-time canonical layout validation
- ? publish-time validation
- ? public render bundle validation
- ? frontend fetch-time validation

WHY MULTI-STAGE VALIDATION IS GOOD

It prevents bad content from:

- ? entering storage unchecked
- ? reaching public rendering silently
- ? drifting across different consumers

ERROR HANDLING STYLE

Errors are structured with:

- ? code
- ? message
- ? optional details
- ? optional trace id

MAIN STATUS CODES USED

- ? '400' invalid request/layout
- ? '401' unauthenticated
- ? '403' forbidden
- ? '404' missing page/version

- ? '409' revision conflict
- ? '500' server/database failure
- ? '503' missing schema/table dependency

ERROR HANDLING WEAKNESS

Page lifecycle flows are better structured than some simpler CRUD domains.

DATABASE DESIGN

CORE TABLES

- ? 'pages'
- ? 'page_revisions'
- ? 'page_templates'
- ? 'headers'
- ? 'footers'
- ? 'banners'
- ? 'navigation_items'
- ? 'media_library'

WHY 'PAGES' AND 'PAGE_REVISIONS' ARE SEPARATE

The page row stores identity and lifecycle pointers. Revisions store versioned content snapshots.

WHY LAYOUT IS STORED AS JSON

- ? nested page-builder structure
- ? fast schema evolution
- ? easy restore/history snapshots

TRADEOFFS OF JSON STORAGE

Pros:

- ? flexible
- ? revision-friendly
- ? simpler page-builder persistence

Cons:

- ? hard to query deeply
- ? app-layer validation required

? less DB-native structure enforcement

WHY GLOBALS ARE SEPARATE TABLES

Headers, footers, banners, and navigation are reusable shared content and should not be duplicated into every page snapshot.

QUERY PATTERNS AND OPTIMIZATION

MAIN READ PATTERNS

- ? page by id
- ? page by slug
- ? revision history by page
- ? current/published revision lookup
- ? global content lookup for render bundle assembly

MAIN WRITE PATTERNS

- ? create revision snapshot
- ? update page revision pointer
- ? publish by creating new published revision
- ? media insert/update

IMPORTANT INDEXING PRIORITIES

- ? page slug lookup
- ? revisions by page id + revision number
- ? revisions by page id + created time
- ? navigation by header id
- ? any frequently joined assignment IDs

OPTIMIZATION OPPORTUNITIES

- ? strict slug-only public lookup
- ? prebuilt published bundles
- ? Redis/shared cache
- ? read-optimized published content model

SECURITY ANALYSIS

MAIN SECURITY WEAKNESSES

- ? simplistic env-based admin auth
- ? no strong RBAC
- ? upload validation not fully hardened
- ? sanitization could be stronger server-side
- ? public/admin runtime coupling increases blast radius
- ? generic CRUD routes can drift into weaker validation

HARDENING IDEAS

- ? DB-backed users and hashed passwords
- ? SSO/OAuth for admin
- ? role-based permissions
- ? audit logs
- ? file signature validation and malware scanning
- ? stronger CSP/security headers
- ? rate limiting
- ? centralized secret management

SCALABILITY ANALYSIS

BIGGEST ARCHITECTURAL RISK

Public content delivery depends on the admin/content runtime at request time.

WHY THAT IS DANGEROUS

Because the same runtime effectively serves:

- ? internal authoring workflows
- ? external public content reads

Those workloads have different scaling and reliability profiles.

READ LOAD CONCERNS

- ? repeated runtime bundle assembly
- ? live dependency on admin service availability
- ? process-local cache only

WRITE LOAD CONCERNS

- ? frequent autosave revisions
- ? large JSON payloads

? revision storage growth

STATELESSNESS CONCERNS

The backend is mostly stateless, but process-local in-memory cache reduces horizontal scaling quality.

BETTER LONG-TERM SCALING DIRECTION

- ? separate published content delivery from admin runtime
- ? precompute published bundles
- ? use shared cache like Redis
- ? possibly introduce CQRS-style read model

DEPLOYMENT, CI/CD, AND OPS

DEPLOYMENT SHAPE

Likely deployments:

- ? admin app
- ? public website app
- ? database

MAIN DEPLOYMENT RISK

Shared contract changes require release coordination between apps.

PRODUCTION CONSIDERATIONS

- ? env validation
- ? startup safety checks
- ? migration sequencing
- ? health checks
- ? rollback strategy

STRONG CI/CD PIPELINE SHOULD INCLUDE

- ? install dependencies
- ? lint/type/build
- ? shared contract tests
- ? migration verification
- ? staging deploy
- ? smoke tests for save/preview/publish/public render

? production promotion

MONITORING SHOULD INCLUDE

- ? page API latency
- ? publish failure rate
- ? validation failures
- ? error rates
- ? cache hit rates

TECHNOLOGY CHOICES AND TRADEOFFS

REACT

Used because:

- ? component-driven UI
- ? strong editor ecosystem
- ? easy integration with Next.js

Alternatives:

- ? Vue
- ? Angular
- ? Svelte

Tradeoff:

- ? very flexible, but requires stronger architectural discipline

NEXT.JS

Used because:

- ? SSR support
- ? route-based app structure
- ? integrated API/backend capabilities

Alternatives:

- ? Remix
- ? Vite + React Router
- ? standalone React + Express

Tradeoff:

? productive and integrated, but framework coupling is higher

MYSQL

Used because:

- ? structured relational data
- ? transactional workflow support
- ? practical and familiar

Alternatives:

- ? PostgreSQL
- ? MongoDB

Tradeoff:

? strong for relational workflows, but JSON-heavy querying is less elegant than PostgreSQL in some cases

NEXTAUTH

Used because:

- ? quick session-based auth integration
- ? protected route support

Alternatives:

- ? custom auth
- ? Auth0
- ? Clerk

Tradeoff:

? reduced auth plumbing, but current usage is too simple for large-scale production needs

DIRECT SQL

Used because:

- ? explicit transaction and query control
- ? clear revision workflow logic

Alternatives:

? Prisma
? Sequelize
? TypeORM

Tradeoff:

? more boilerplate, but fewer ORM abstraction surprises

SMALL ANSWER BANK

WHAT PROBLEM DOES THE PROJECT SOLVE?

It gives internal users a safe way to manage and publish website content with drafts, preview, revisions, and a clear separation between editing and live delivery.

WHAT IS THE STRONGEST TECHNICAL DECISION?

The revision-based content lifecycle combined with shared validation.

WHAT IS THE BIGGEST WEAKNESS?

Public page delivery still depends on the admin/content service at runtime.

WHY IS THE SHARED LAYER IMPORTANT?

Because it prevents preview and production from interpreting content differently.

WHY USE SSR ON THE PUBLIC SITE?

For SEO, first-response correctness, and content-driven rendering.

WHY STORE LAYOUTS AS JSON?

Because page-builder content is nested and changes shape over time.

WHY USE REVISIONS?

To support draft isolation, restore, preview, publish, and safer history tracking.

WHY NOT JUST UPDATE THE PAGE ROW DIRECTLY?

Because live content should not be overwritten by in-progress edits.

WHY NOT LET THE FRONTEND SHAPE CMS DATA ITSELF?

Because that would create multiple sources of truth for content interpretation.

HOW DO YOU HANDLE CONCURRENT EDITS?

Optimistic concurrency using revision baselines.

LONG ANSWER BANK

EXPLAIN THE PROJECT END-TO-END

The system is a CMS-style web platform. Admin users authenticate into an authoring application and edit pages through a visual page builder. The editor stores content as structured layout data composed of sections, rows, columns, and blocks. Draft changes are autosaved and manually saveable as revisions rather than being applied directly to the live site. When a page is previewed, the system loads draft revision data and renders it through the shared layout and bundle pipeline. When a page is published, the current draft is validated for public use, stored as a published revision, and referenced as the live version of that page. On the read side, the public website requests the page bundle by slug, the backend assembles page metadata, the published revision, and any global content like header/footer/banner, validates the canonical render bundle, and the website server-renders the result. The design intentionally separates writing, interpretation, and rendering responsibilities to reduce drift and improve lifecycle safety.

EXPLAIN THE FRONTEND ARCHITECTURE END-TO-END

The admin frontend is a rich React editor built around local state and custom hooks. It loads revision-backed layout data, renders it dynamically through a registry, and supports interactions like drag-and-drop, property editing, autosave, undo/redo, preview, version history, and template application. Its biggest strength is workflow grouping through hooks, and its biggest weakness is orchestration density in the top-level editor container. The public frontend is much thinner. It uses SSR to fetch a validated page bundle, derive a public page view, and render a stable page shell. The public app intentionally avoids becoming a content interpretation layer.

EXPLAIN THE BACKEND ARCHITECTURE END-TO-END

The backend is built around Next.js route handlers, services, repositories, shared validation, and MySQL persistence. Sensitive routes are protected through middleware and server-side auth checks. Save and publish workflows are handled in services, not in raw route handlers, because they involve validation, transactions, revision creation, and page-

pointer updates. The public content API builds a canonical render bundle rather than returning raw DB records. This gives the frontend a stable rendering contract while keeping content meaning centralized. The strongest part of the backend is the revision-first page pipeline. The biggest architectural weakness is that public delivery still depends on the authoring runtime at request time.

TOUGH INTERVIEW QUESTIONS WITH STRONG DIRECTION

WHY IS RUNTIME DEPENDENCY BETWEEN PUBLIC SITE AND ADMIN DANGEROUS?

Because public traffic becomes dependent on the same backend that serves authoring workflows. That creates availability coupling, scaling tension, and operational risk. If public traffic spikes, it can affect content editors. If admin/content assembly slows down, it directly impacts public latency. A stronger design would separate published-content delivery into a read-optimized path or precomputed artifact.

WHY DIDN'T YOU JUST USE WORDPRESS OR A NORMAL CMS?

Because the main challenge wasn't just content fields, it was lifecycle control and render contract consistency. I wanted revision-based draft/preview/publish flow and a shared interpretation layer that keeps preview and production aligned.

IF TRAFFIC BECAME 100X LARGER, WHAT WOULD YOU CHANGE FIRST?

I would first decouple public content delivery from the admin runtime. That gives the biggest immediate improvement to scalability and public reliability. After that, I'd add shared caching, read-optimized published bundles, and stronger observability.

WHAT WOULD YOU IMPROVE FIRST FOR PRODUCTION SECURITY?

I would replace the current admin-only auth model with real users, hashed credentials or SSO, stronger role-based authorization, and audit logging. I would also harden uploads and content sanitization.

WHAT WOULD YOU IMPROVE FIRST FOR FRONTEND MAINTAINABILITY?

I would split the editor shell into smaller state-owning modules and improve lazy loading for heavy editor features.

TECHNOLOGY CHOICE Q&A

WHY REACT?

React was the right fit because the admin experience is deeply interactive and component-driven, and the public site also benefits from reusable rendering components. Vue or Angular could also have worked, but React plus Next.js gave a strong ecosystem fit for this project.

WHY NEXT.JS?

Next.js gave SSR for the public site, route-based structure, and integrated backend capabilities through route handlers. That helped unify the frontend/backend development model.

WHY NODE.JS?

Using Node.js let the team use the same language across backend and frontend concerns, which simplifies shared contracts and reduces context switching.

WHY MYSQL?

MySQL is a practical choice for structured content relationships and transactions. It handles page metadata, revision pointers, and related entities well. The tradeoff is that deeply querying layout JSON is less elegant than in some other databases.

WHY NEXTAUTH?

NextAuth reduced the amount of custom session/auth infrastructure needed. For an internal admin setup, it was a reasonable speed-vs-complexity tradeoff.

100 HIGH-PROBABILITY QUESTIONS AND ANSWERS

1. WHAT PROBLEM DOES THIS PROJECT SOLVE?

It solves safe content management for a website by separating draft editing from live content and adding preview, publish, and revision history.

2. WHAT ARE THE MAJOR PARTS OF THE SYSTEM?

The major parts are the admin authoring app, the shared content contract layer, and the public SSR website.

3. WHY SPLIT ADMIN AND PUBLIC?

Because editing workflows and public rendering have different concerns and should not share the same runtime responsibilities too tightly.

4. WHY IS THE SHARED LAYER NECESSARY?

It gives one canonical interpretation of blocks and layouts so preview and public rendering

stay aligned.

5. HOW DOES A PAGE MOVE FROM DRAFT TO LIVE?

An editor saves draft revisions, previews them, and publishes the selected draft as the current live revision. The public site only reads the published revision.

6. WHY USE REVISIONS?

Revisions support autosave, restore, preview, rollback, and safe publishing.

7. WHY NOT UPDATE CONTENT DIRECTLY?

Because editing in-progress content directly into live state is risky and removes safe rollback boundaries.

8. WHY STORE LAYOUTS AS JSON?

Because page-builder structures are nested and change over time.

9. WHY SSR?

Because SEO and first-response correctness matter for public content pages.

10. WHY USE ROUTE HANDLERS INSTEAD OF A SEPARATE BACKEND AT FIRST?

Because it was faster to build and aligned well with the project's full stack structure.

11. WHAT IS THE BIGGEST BACKEND WEAKNESS?

Public delivery depending on the admin/content runtime.

12. WHAT IS THE BIGGEST FRONTEND WEAKNESS?

The editor shell owns too much orchestration and state.

13. WHAT IS THE BIGGEST SECURITY WEAKNESS?

Auth and authorization are too simple for broader production use.

14. HOW DO YOU PREVENT INVALID CONTENT FROM REACHING PRODUCTION?

Through save-time and render-time validation.

15. WHY VALIDATE MORE THAN ONCE?

Because data can drift, migrate, or become invalid after persistence too.

16. WHAT HAPPENS IF TWO USERS EDIT THE SAME PAGE?

Optimistic concurrency causes the stale save to fail rather than silently overwrite.

17. WHY USE OPTIMISTIC CONCURRENCY?

Because it's simpler than locking and works well when conflicts are uncommon.

18. WHAT ARE THE MAIN DB TABLES?

Pages, page revisions, templates, globals, navigation, and media.

19. WHAT IS THE ROLE OF THE PAGE TABLE?

Identity, metadata, and pointers to revision states.

20. WHAT IS THE ROLE OF THE REVISIONS TABLE?

Immutable snapshots of layout content over time.

21. WHY SEPARATE GLOBALS LIKE HEADER AND FOOTER?

Because they are reusable and should not be duplicated per page.

22. HOW IS NAVIGATION MODELED?

Through navigation items, which are later assembled into a tree.

23. HOW DOES THE EDITOR STATE WORK?

Local state plus custom hooks for page loading, mutation, autosave, and history.

24. WHY NOT REDUX?

At this stage, local workflow hooks were simpler and faster to iterate with.

25. WHY USE A REGISTRY FOR BLOCKS?

To make block rendering extensible and centralized.

26. WHAT IS THE HARDEST FRONTEND PROBLEM HERE?

Nested layout state management with drag-and-drop plus persistence and history.

27. WHAT IS THE HARDEST BACKEND PROBLEM HERE?

Maintaining a clean canonical content contract across save, preview, publish, and public render.

28. HOW DO YOU RENDER PAGES PUBLICLY?

Fetch the bundle, validate it, derive a public view, SSR the final page.

29. WHY NOT LET THE FRONTEND FIX BAD CMS DATA?

Because that creates another interpretation layer and increases drift risk.

30. WHAT WOULD YOU IMPROVE FIRST AT SCALE?

Separate published-content delivery from admin runtime.

31. WHAT WOULD YOU IMPROVE FIRST FOR ENTERPRISE READINESS?

User system, RBAC, audit logging, and stronger auth.

32. WHY USE MYSQL OVER MONGODB?

Because the workflow is still strongly relational even though layouts are JSON-like.

33. WHY NOT POSTGRESQL?

PostgreSQL would also have been a strong choice; MySQL was still reasonable for the current constraints.

34. WHAT IS ONE GOOD REASON FOR DIRECT SQL?

Explicit control over revision and transaction behavior.

35. WHAT IS ONE DOWNSIDE OF DIRECT SQL?

More manual query and schema maintenance.

36. WHY USE NEXTAUTH?

To reduce custom auth plumbing and integrate sessions quickly.

37. WHAT IS WRONG WITH ENV-BASED ADMIN CREDENTIALS?

They do not scale operationally to real multi-user, secure production use.

38. WHAT IS YOUR LOGGING STRATEGY TODAY?

Basic logs plus structured error boundaries, but observability should be improved.

39. WHAT OBSERVABILITY WOULD YOU ADD?

Structured logs, metrics, alerts, and request correlation IDs.

40. HOW WOULD YOU CACHE PUBLIC CONTENT?

Use Redis or precomputed published bundles rather than process-local memory.

41. WHY IS IN-MEMORY CACHE WEAK?

Because it does not scale horizontally and is instance-local.

42. WHAT IS THE ROLE OF TEMPLATES?

Reusable layout starting points for faster content creation.

43. WHAT IS THE ROLE OF AUTOSAVE?

Protect user work without requiring constant manual saves.

44. WHAT IS THE DOWNSIDE OF AUTOSAVE?

More write traffic and more concurrency complexity.

45. WHAT HAPPENS DURING PUBLISH FAILURE?

The API returns a clear error and the public site remains unchanged because published state did not advance.

46. HOW IS PREVIEW PROTECTED?

Through admin access control and preview-mode content resolution.

47. WHAT IS THE ROLE OF THE SHARED PAGE RENDER BUNDLE?

It is the canonical payload that bridges backend content assembly and public rendering.

48. WHAT IS THE ROLE OF SCHEMA VERSIONING IN SHARED CONTENT?

It gives a path for evolving content contracts more safely.

49. WHAT IS THE ROLE OF NORMALIZATION?

To convert flexible incoming/editor content shapes into one canonical structure.

50. WHY IS CANONICAL STRUCTURE IMPORTANT?

Because rendering and validation become much simpler when the shape is consistent.

51. WHAT WOULD YOU SAY IS THE SYSTEM'S BEST DESIGN CHOICE?

Revision-first page lifecycle plus shared validation.

52. WHAT WOULD YOU SAY IS THE SYSTEM'S WEAKEST DESIGN CHOICE?

Public request-time dependency on the admin/content runtime.

53. HOW WOULD YOU EXPLAIN THE PROJECT TO A NON-TECHNICAL STAKEHOLDER?

It's a system that lets the team safely edit and publish website content without directly touching live pages.

54. HOW WOULD YOU EXPLAIN IT TO A SENIOR ENGINEER?

It's a revision-based CMS platform with a shared canonical content layer and SSR public rendering.

55. WHY USE CUSTOM HOOKS IN THE EDITOR?

To organize workflow logic like save, history, and layout mutation.

56. WHAT IS STILL TOO BIG IN THE FRONTEND?

The unified page editor shell.

57. WHAT IS STILL TOO COUPLED IN THE BACKEND?

Public content delivery and admin runtime.

58. WHAT IS THE ROLE OF MIDDLEWARE?

To protect route groups and improve admin navigation security flow.

59. WHY DO APIS STILL CHECK AUTH IF MIDDLEWARE EXISTS?

Because APIs are the actual server-side trust boundary.

60. HOW ARE MEDIA UPLOADS STORED?

Locally in uploads plus metadata in the media library.

61. WHAT IS THE DOWNSIDE OF LOCAL UPLOAD STORAGE?

Weaker scalability and durability than object storage.

62. WHAT WOULD BE BETTER LATER FOR UPLOADS?

Object storage like S3 or Cloudinary-style delivery.

63. WHAT IS THE ROLE OF TEMPLATES IN SCALING CONTENT OPERATIONS?

They reduce repeated editor effort and standardize starting structures.

64. WHAT IS THE ROLE OF PAGE HISTORY IN OPERATIONAL SAFETY?

It allows recovery and rollback when content changes go wrong.

65. WHAT WOULD YOU CHANGE FIRST IN THE DATABASE AT LARGER SCALE?

Introduce a read-optimized published content model or cache layer.

66. WHAT IS THE FRONTEND'S STRONGEST BOUNDARY?

Public rendering staying thin and contract-driven.

67. WHAT IS THE BACKEND'S STRONGEST BOUNDARY?

Canonical validation around page lifecycle workflows.

68. WHAT COULD GO WRONG DURING DEPLOY?

Contract mismatch, schema mismatch, or public rendering breakage if releases are not compatible.

69. HOW WOULD YOU REDUCE DEPLOY RISK?

Backward-compatible changes, contract tests, staging smoke tests.

70. WHAT WOULD YOUR SMOKE TEST COVER?

Create/edit/save/preview/publish/public render.

71. WHAT IS THE VALUE OF A SHARED REPO HERE?

Easier refactors and coordinated evolution across apps and contracts.

72. WHAT IS THE RISK OF A SHARED REPO?

Boundaries can blur if discipline is weak.

73. WHY IS PAGE CONTENT NOT JUST PLAIN HTML STORAGE?

Because structured blocks are more reusable, safer, and easier to validate.

74. WHEN WOULD PLAIN HTML STORAGE BE TEMPTING?

For very simple publishing systems without complex block semantics.

75. WHY KEEP PUBLIC FRONTEND THIN?

To avoid duplication of content meaning.

76. WHY KEEP ADMIN BACKEND WORKFLOW-RICH?

Because content lifecycle safety requires explicit workflow handling.

77. WHAT WOULD YOU SAY IF ASKED 'DID YOU OVERENGINEER THIS'?

I'd say the complexity is deliberate around the page lifecycle, because content publishing is where silent failure is most costly.

78. WHAT PARTS ARE MAYBE UNDERENGINEERED?

Auth, RBAC, observability, and public delivery scaling.

79. WHAT MAKES THIS A STRONG INTERVIEW PROJECT?

It shows system design thinking across frontend, backend, data modeling, and content lifecycle.

80. WHAT MAKES IT VULNERABLE IN INTERVIEWS?

If I overclaim production readiness in auth, RBAC, or scaling.

81. WHAT SHOULD YOU BE HONEST ABOUT?

That the page architecture is stronger than some surrounding domains and that some parts are still evolving.

82. WHAT ARE THE TOP FUTURE IMPROVEMENTS?

Published-content delivery separation, auth/RBAC hardening, editor modularization.

83. WHAT IS THE RIGHT WAY TO EXPLAIN THE BIGGEST TRADEOFF?

We accepted more application complexity to get safer content lifecycle and shared rendering correctness.

84. WHAT'S THE WRONG WAY TO EXPLAIN THE TRADEOFF?

Pretending there were no downsides or that the current system is fully enterprise-grade.

85. WHY DO INTERVIEWER FOLLOW-UPS HAPPEN SO MUCH ON THIS PROJECT?

Because the architecture makes strong claims around workflow, consistency, and lifecycle safety.

86. WHAT IS ONE SIGN YOU REALLY UNDERSTAND THE PROJECT?

You can explain the exact difference between save, preview, and publish.

87. WHAT IS ANOTHER SIGN?

You can explain why the shared layer exists and what would break without it.

88. WHAT IS ANOTHER SIGN?

You can explain why public rendering should not repair CMS data.

89. WHAT IS ANOTHER SIGN?

You can explain why revisions are better than updating live state directly.

90. WHAT IS ANOTHER SIGN?

You can explain what happens when two editors collide.

91. WHAT SHOULD YOU SAY IF ASKED ?WHY NOT WORDPRESS??

Because the core need was controlled block-based lifecycle and rendering semantics, not just simple content entry.

92. WHAT SHOULD YOU SAY IF ASKED ?WHY NOT GRAPHQL??

Because the current consumers are known and the workflow-heavy API was simpler with REST-like routes.

93. WHAT SHOULD YOU SAY IF ASKED ?WHY NOT PRISMA??

Direct SQL gave explicit control over revision and transactional workflows, though Prisma could be a valid future option.

94. WHAT SHOULD YOU SAY IF ASKED ?WHY NOT MONGODB??

Even though layouts are JSON-like, the rest of the system is still strongly relational and transactional.

95. WHAT SHOULD YOU SAY IF ASKED ?WHY NOT STATIC SITE GENERATION??

It would be attractive later, but dynamic preview and early workflow speed were stronger initial priorities.

96. WHAT SHOULD YOU SAY IF ASKED ?WHAT FAILS FIRST AT SCALE??

Public runtime coupling to admin/content service.

97. WHAT SHOULD YOU SAY IF ASKED ?WHAT FAILS FIRST SECURITY-WISE??

The simplistic auth/authorization model.

98. WHAT SHOULD YOU SAY IF ASKED ?WHAT FAILS FIRST FRONTEND-WISE??

Editor maintainability and render isolation on large pages.

99. WHAT SHOULD YOU SAY IF ASKED ?WHAT ARE YOU MOST PROUD OF??

The revision-first content lifecycle and shared contract direction.

100. WHAT SHOULD YOU SAY IF ASKED ?WHAT WOULD YOU DO NEXT??

Decouple published delivery, improve auth/RBAC, and modularize the editor further.

FINAL ADVICE

The interview goal is not to sound perfect.

The goal is to sound:

? structured

? honest

? technically deep

? aware of tradeoffs

? aware of what should happen next

The strongest pattern is:

1. what it does
2. why it exists
3. tradeoff
4. future improvement

If you keep answering in that structure, you will sound much more senior.